Name: Christina.y
Reg No:
1965260124

1. 8-PUZZLE

Aim

To implement the 8-Puzzle problem using Python and move the tiles to reach the goal state.

Algorithm

1. Start with the initial puzzle.
2. Set the goal as 1 2 3 4 5 6 7 8 0.
3. Find the position of 0.
4. Check whether the selected movement is
valid. 5. Swap the blank with the adjacent tile.
6. Repeat until the puzzle is solved.

Program

```
p = [1, 2, 3, 4, 0, 6, 7, 5, 8]

goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

print("Initial:")
print(p[0], p[1], p[2])
print(p[3], p[4], p[5])
print(p[6], p[7], p[8])

while p != goal:

    pos = p.index(0)

    print("\n0 = Blank")
    print("1. Up")
    print("2. Down")
    print("3. Left")
    print("4. Right")

    choice = int(input("Enter move: "))

    if choice == 1 and pos >= 3:
        p[pos], p[pos-3] = p[pos-3], p[pos]

    elif choice == 2 and pos < 6:
        p[pos], p[pos+3] = p[pos+3], p[pos]

    elif choice == 3 and pos % 3 != 0:
        p[pos], p[pos-1] = p[pos-1], p[pos]

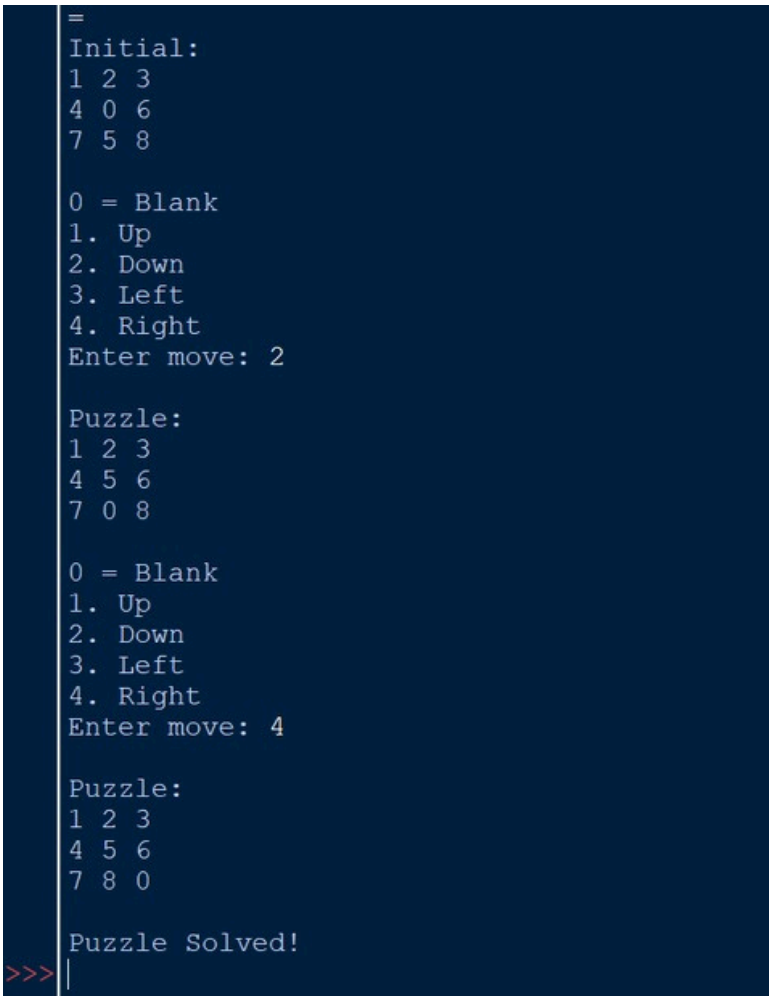
    elif choice == 4 and pos % 3 != 2:
        p[pos], p[pos+1] = p[pos+1], p[pos]

    else:
        print("Invalid move")
        continue
```

```
    print("\nPuzzle:")
    print(p[0], p[1], p[2])
    print(p[3], p[4], p[5])
    print(p[6], p[7], p[8])

print("\nPuzzle Solved!")
```

Output



Result

Thus the 8-Puzzle problem was successfully implemented and solved.

2. 8-QUEEN

Aim

Toimplement the 8-Queen problem using Python by placing eight queens on a chessboard without attacking each other.

Algorithm

1. Initializean empty 8×8 board.
2. Start placing a queen row by row.
3. Take the column position as input.
4. Check for same-column and diagonal conflicts. 5. Place the queen if the position is safe.
6. Repeat until all 8 queens are placed.

Program

```
n = 8
board = [-1] * n

for row in range(n):

    print("\nBoard:")

    for i in range(n):
        for j in range(n):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

    col = int(input("Enter column (1-8) for Queen: ")) - 1

    safe = True

    for i in range(row):
        if board[i] == col:
            safe = False

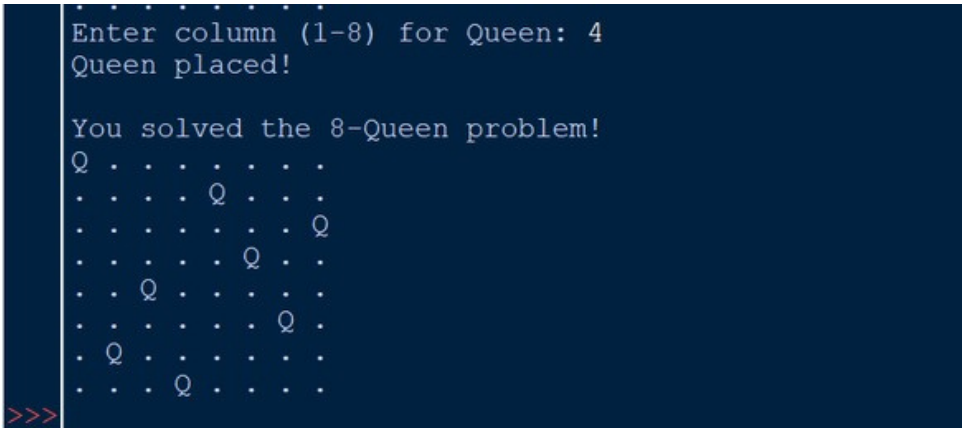
        if abs(board[i] - col) == abs(i - row):
            safe = False

    if safe:
        board[row] = col
        print("Queen placed!")
    else:
        print("Invalid position! Try again.")
        continue

print("\nYou solved the 8-Queen problem!")
```

```
for i in range(n):
    for j in range(n):
        if board[i] == j:
            print("Q", end=" ")
        else:
            print(".", end=" ")
    print()
```

Output



Result

Thus, the 8-Queen problem was successfully implemented by placing queens without attacking each other.

3. WATER JUG

Aim

To implement the Water Jug problem using Python and obtain the required quantity of water.

Algorithm

1. Initialize both jugs to zero.
2. Set the capacities as 4L and 3L. 3.

Select an operation.

4. Perform the selected operation.
5. Check whether 2L is obtained.
6. Repeat until the goal is reached.

Program

```
a = 0
b = 0

print("Jug 1 = 4 litres")
print("Jug 2 = 3 litres")
print("Goal = 2 litres")

while a != 2 and b != 2:

    print("\n1. Fill Jug 1")
    print("2. Fill Jug 2")
    print("3. Empty Jug 1")
    print("4. Empty Jug 2")
    print("5. Pour Jug 1 into Jug 2")
    print("6. Pour Jug 2 into Jug 1")

    choice = int(input("Enter choice: "))

    if choice == 1:
        a = 4

    elif choice == 2:
        b = 3

    elif choice == 3:
        a = 0

    elif choice == 4:
        b = 0

    elif choice == 5:
        x = min(a, 3 - b)
        a = a - x
        b = b + x

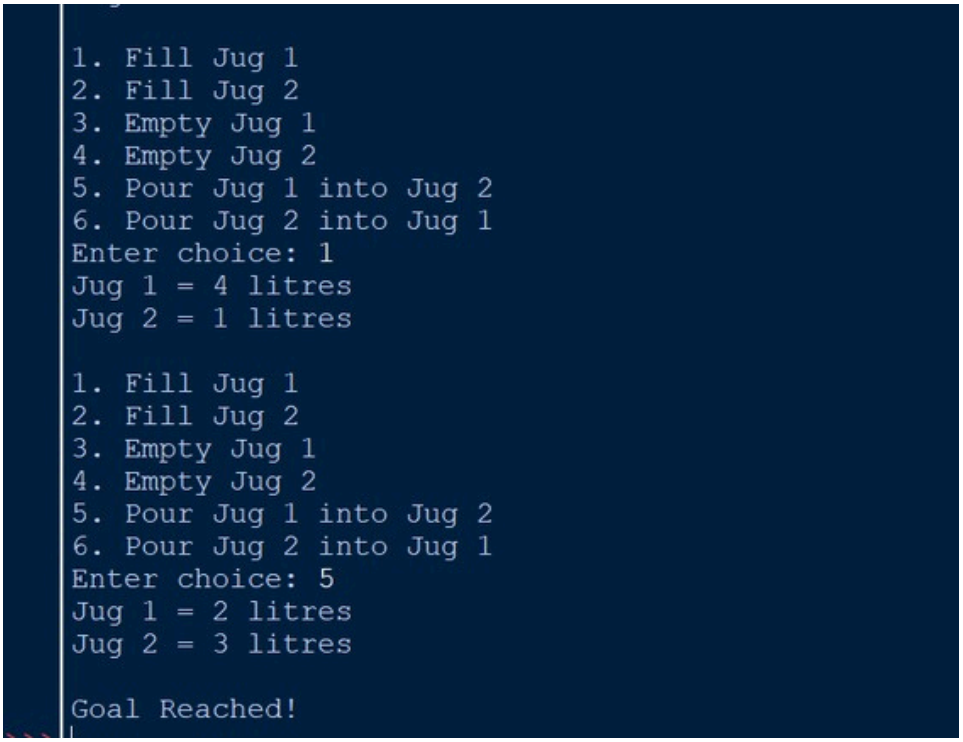
    elif choice == 6:
        x = min(b, 4 - a)
        b = b - x
        a = a + x

    else:
        print("Invalid choice")

    print("Jug 1 =", a, "litres")
    print("Jug 2 =", b, "litres")

print("\nGoal Reached!")
```

Output



Result

Thus,the Water Jug problem was successfully implemented and the required 2 litres of water was obtained.

4. CRYPT-ARITHMETIC

Aim

To implement a Crypt-Arithmetic problem in Python where each letter represents a unique digit.

Algorithm

1. Startwith the letters S, E, N, D, M, O, R, Y.
2. Try different digits for each letter.
3. Check that all digits are different.
4. Form the numbers SEND, MORE and MONEY.
5. Check whether SEND + MORE = MONEY.
6. Display the solution.

Program

```
for S in range(1, 10):
    for E in range(10):
        for N in range(10):
            for D in range(10):
                for M in range(1, 10):
                    for O in range(10):
                        for R in range(10):
                            for Y in range(10):

                                if len({S,E,N,D,M,O,R,Y}) == 8:

                                    SEND = 1000*S + 100*E + 10*N + D
                                    MORE = 1000*M + 100*O + 10*R + E
                                    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

                                    if SEND + MORE == MONEY:
                                        print("SEND =", SEND)
                                        print("MORE =", MORE)
                                        print("MONEY =", MONEY)
                                        print("Solution Found")
```

Output

```
>>>
========== RESTART: D:\SLOT-2\AI\exp4.py ==========
SEND = 9567
MORE = 1085
MONEY = 10652
Solution Found
>>>
```

Result

Thus,the Crypt-Arithmetic problem was successfully solved by assigning unique digits to the letters.

5. MISSIONARIES AND CANNIBALS

Aim

To implement the Missionariesand Cannibals problem using Python and find a valid sequence of states to move all people across the river.

Algorithm

1. Set the initial state as (3,3,1).
2. Generate possible combinations of missionaries and cannibals. 3. Move them according to the boat position.
4. Check that missionaries are never outnumbered by cannibals. 5. Continue until (0,0,0) is reached.
6. Display the solution path.

Program

```
states = [(3, 3, 1)]
parent = {}

i = 0

while i < len(states):

    m, c, boat = states[i]
    i = i + 1

    for x in range(3):
        for y in range(3):

            if x + y >= 1 and x + y <= 2:

                if boat == 1:
                    nm = m - x
                    nc = c - y
                    nb = 0
                else:
                    nm = m + x
                    nc = c + y
                    nb = 1

                if nm >= 0 and nc >= 0 and nm <= 3 and nc <= 3:

                    if (nm == 0 or nm >= nc) and (3-nm == 0 or 3-nm >= 3-nc):

                        new = (nm, nc, nb)

                        if new not in states:
                            states.append(new)
                            parent[new] = (m, c, boat)

    if (0, 0, 0) in states:
        break
```

```
current = (0, 0, 0)
path = []

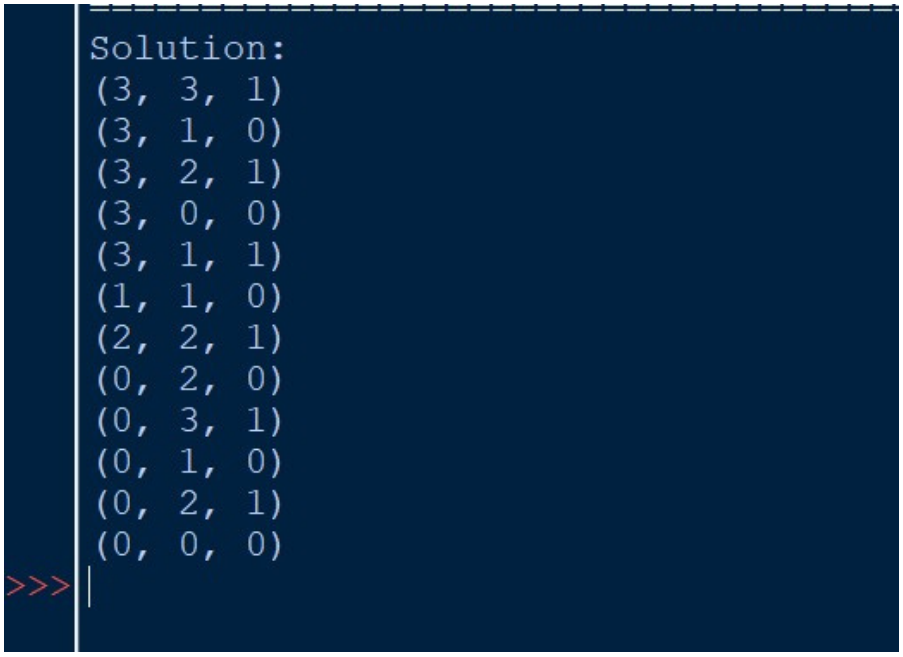
while current != (3, 3, 1):
    path.append(current)
    current = parent[current]

path.append((3, 3, 1))
path.reverse()

print("Solution:")

for x in path:
    print(x)
```

Output



Result

Thus, the Missionaries and Cannibals problem was successfully implemented and a valid solution path was obtained.